

PRÁCTICA 1: PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Juan Sebastian Bonces Junco - 2234562

Santiago Benitez Torres - 221752

Óscar Germán Hernández Salgado - 2212260

https://github.com/Elbonces/2026_2_CommII_A1_GI

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones

Universidad Industrial de Santander

1 de septiembre de 2026

Abstract—In this laboratory work, custom digital signal processing (DSP) blocks were implemented and evaluated in GNU Radio Companion using Embedded Python Blocks. The performance of a discrete-time accumulator was analyzed under square wave inputs and variable sampling rates, alongside a time-domain differentiator applied to square, constant, and ramp waveforms for transition and edge detection. Furthermore, a real-time statistical analyzer was designed to estimate the mean, RMS value, average power, and standard deviation. The system's behavior was validated against theoretical models under additive Gaussian noise, and an application for detecting signal degradation in a communication channel was evaluated. The experimental findings confirm the numerical accuracy of the programmed algorithms and the versatility of Software-Defined Radio platforms.

Index Terms—GNU Radio, Embedded Python Blocks, Digital Signal Processing, Accumulator, Differentiator, Statistical Analysis, Software-Defined Radio.

I INTRODUCCIÓN

La Radio Definida por Software (SDR) ha tomado gran importancia en las telecomunicaciones debido a que permite reemplazar etapas de procesamiento en hardware mediante algoritmos ejecutados en software [1]. En este entorno, plataformas de código abierto como GNU Radio facilitan la simulación y el procesamiento de señales en tiempo real mediante diagramas de flujo interactivos [2].

A pesar de que GNU Radio cuenta con una amplia variedad de bloques estándar, muchas aplicaciones requieren el desarrollo de funciones matemáticas a la medida. Para esto, los bloques embebidos de Python (*Embedded Python Blocks*) permiten programar lógica propia de procesamiento digital de señales (DSP) de forma rápida, evitando tener que compilar módulos en C++ [2].

En este laboratorio se diseñaron, implementaron y

evaluaron experimentalmente tres algoritmos en GNU Radio: un acumulador discreto para analizar su memoria recursiva, un diferenciador temporal para la detección de flancos y cambios de amplitud, y un módulo para el cálculo de parámetros estadísticos en tiempo real. Adicionalmente, se analizó la respuesta del sistema frente a señales contaminadas con ruido gaussiano [3] y se evaluó una aplicación orientada al monitoreo de la degradación en un canal de comunicaciones.

II METODOLOGÍA

El propósito de la práctica consistió en implementar cuatro propuestas de diseño, fundamentadas en el desarrollo de bloques funcionales y en la validación estadística de las señales.

II-A Diseño e Implementación de bloques

Para el desarrollo de la práctica en GNU Radio, se utilizaron bloques de Python embebidos (*Embedded Python Blocks*), los cuales permitieron programar la lógica interna requerida para la implementación de:

- **Acumulador:** Corresponde al algoritmo diseñado para sumar de forma acumulativa las muestras de entrada, facilitando los procesos de recuperación de señales:

$$y[n] = \sum_{k=0}^n x[k] \quad (1)$$

- **Diferenciador:** Corresponde al algoritmo que resalta los cambios o transiciones rápidas de amplitud a través de la diferencia entre muestras sucesivas:

$$y[n] = x[n] - x[n - 1] \quad (2)$$

- **Operadores Estadísticos:** Se implementaron algoritmos orientados a la extracción en tiempo real de

diversas métricas, tales como el valor eficaz (RMS), la media (μ_x), la desviación estándar (σ_x) y la potencia promedio (P).

III RESULTADOS

Se estudió el funcionamiento de los bloques dentro de GNU Radio a través de la aplicación de algoritmos de diferenciación, acumulación y evaluación estadística.

III-A Acumulador

III-A1. Señal de entrada cuadrada

El acumulador actúa como integrador en tiempo discreto, suavizando cambios bruscos y compensando ruido al sumar muestras. Se evaluó con una señal cuadrada a 32 kHz, un *Vector Source* de 8 muestras y un *Throttle* (ver Figura 1). La salida obtenida tuvo forma de rampa, confirmando el correcto funcionamiento del bloque como integrador.

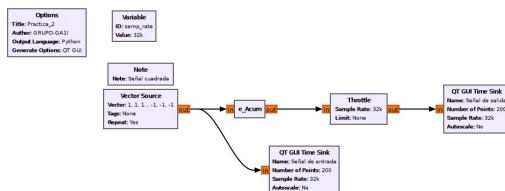


Figura 1: Diagrama de bloques del acumulador.

Análisis de señal: La obtención de una salida con forma de rampa evidencia el correcto funcionamiento del bloque como un integrador en tiempo discreto (Figura 2).

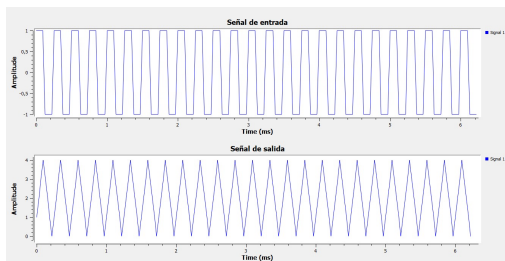


Figura 2: Señal de entrada y salida.

III-A2. Señal de entrada cuadrada, variando el número de muestras

Se duplicó el número de muestras de 8 a 16, manteniendo 32 kHz. La salida conservó la forma de rampa, con el doble de amplitud (Figura 3), confirmando que el acumulador es recursivo y que su estado interno depende del número de muestras con el mismo signo, no de la amplitud de entrada.

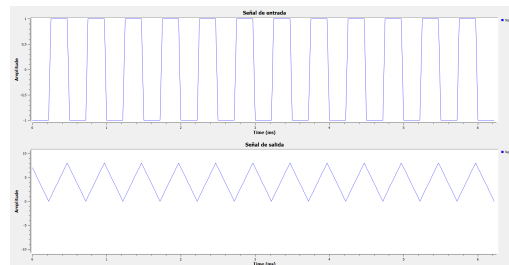


Figura 3: Señal de entrada y salida del acumulador con 16 muestras.

III-A3. Señal de entrada cuadrada, variando la frecuencia de muestreo

Se redujo la frecuencia de muestreo de 32 kHz a 2 kHz, sin cambiar el *Vector Source*. La salida no cambió en forma ni amplitud, confirmando la dependencia del número de muestras, aunque cada ciclo tardó más en completarse, evidenciando menor resolución temporal (Figura 4).

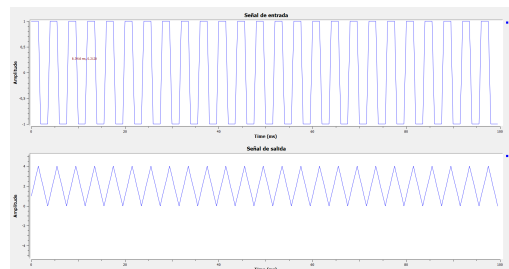


Figura 4: Señal de entrada y salida con frecuencia de muestreo de 2 kHz.

III-B Diferenciador

Corresponde a un sistema diseñado para procesar la señal de entrada y resaltar las transiciones o cambios rápidos en su amplitud (ver Figura 5).

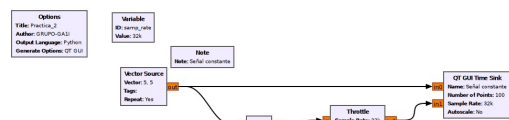


Figura 5: Diagrama de bloques del diferenciador.

III-B1. Diferenciador con entrada constante

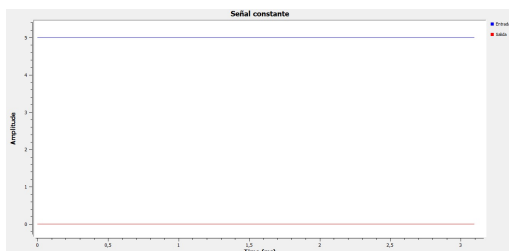


Figura 6: Respuesta ante una entrada constante.

Como se observa en la Figura 6, la señal de salida toma únicamente valores de cero. Esto ocurre porque, al no registrarse variaciones en la amplitud de entrada, el bloque efectúa de forma constante la resta entre la muestra actual y la anterior ($5 - 5$), confirmando que el diferenciador responde exclusivamente ante cambios en la señal.

III-B2. Diferenciador con entrada rampa

Nuestra señal consta de una entrada en forma de rampa con magnitud máxima de 6 y con pendiente 1.

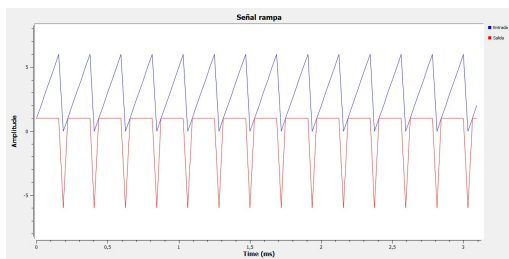


Figura 7: Respuesta ante una entrada rampa.

Con una entrada tipo rampa (Figura 7), la salida se estabiliza en un valor constante de 1, debido a la pendiente unitaria fija entre muestras. Al alcanzar el valor máximo (6) y reiniciarse a 0, el diferenciador genera un pico de amplitud -6 , al restar el valor mínimo del máximo.

III-B3. Diferenciador con entrada cuadrada

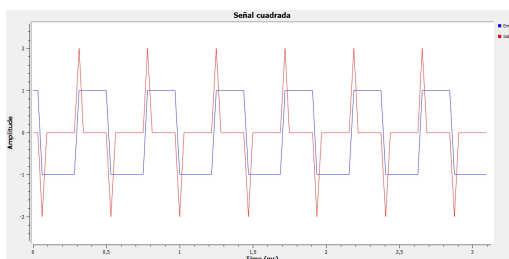


Figura 8: Respuesta ante una entrada cuadrada.

En la Figura 8 se evidencia que en la salida aparecieron picos claros en las transiciones: uno positivo al subir

y uno negativo al bajar. Estos picos reflejan el cambio abrupto entre muestras consecutivas, permitiendo identificar exactamente los flancos de subida y bajada de la señal de entrada.

III-C Análisis Estadístico en Tiempo Real

Se implementó un diagrama de flujo mediante un *Embedded Python Block* que calcula en tiempo real la media, media cuadrática, RMS, potencia promedio y desviación estándar de una señal de entrada, usando sumas acumulativas (*cumsum*) que actualizan cada métrica muestra a muestra sin necesidad de almacenar el historial completo (Figura 9).

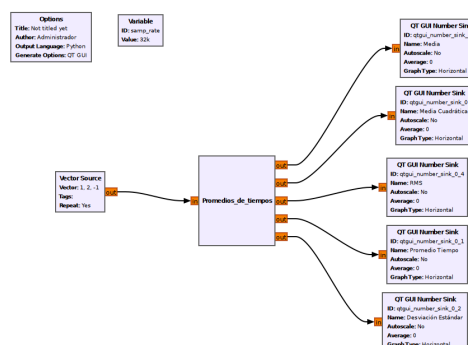


Figura 9: Diagrama de flujo del bloque de análisis estadístico.

Para la validación inicial se usó un *Vector Source* con $(1, 2, -1)$ a 32 kHz. Los valores obtenidos (Media = 0,6667, Media Cuadrática = 2,0, RMS = 1,4142, Potencia = 2,0, Desv. Est. = 1,2472) coincidieron con los teóricos. La potencia promedio y la media cuadrática arrojaron el mismo valor, consistente con la teoría para señales reales.

III-C1. Validación bajo ruido

Se construyó un segundo diagrama de flujo donde una señal cuadrada de amplitud ± 1 se suma con un *Noise Source* gaussiano antes de ingresar al analizador estadístico, evaluando su robustez con ruido de amplitud 0 y 0.1 (Figura 10).

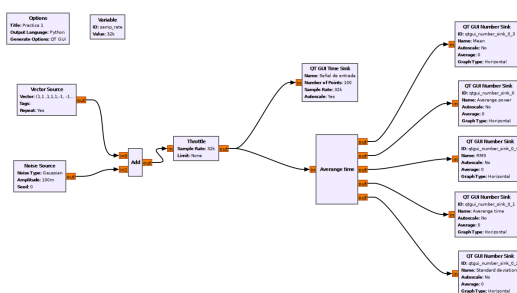


Figura 10: Validación del analizador estadístico bajo ruido gaussiano.

Tabla I: Validación de bloques: teórico vs. ejecución bajo ruido.

Métrica	Ruido = 0	Ruido = 0.1
Media (teo.)	0	≈ 0
Media (obt.)	0.000244	0.193174
RMS (teo.)	1	$\approx 1,005$
RMS (obt.)	1.000000	1.007665
Potencia (teo.)	1	$\approx 1,01$
Potencia (obt.)	1.000000	1.015388
Desv. Est. (teo.)	1	$\approx 1,005$
Desv. Est. (obt.)	1.000000	0.988975

Con ruido de amplitud 0, los valores coincidieron con los teóricos para una señal cuadrada unitaria. Con amplitud 0.1, el RMS, la potencia y la desviación estándar se mantuvieron estables, mientras que la media osciló debido a errores de redondeo en sumas acumuladas de precisión float32.

Tabla II: Resultados estadísticos con variaciones de vector, F_s y N .

Nº Muest.	F_s [kHz]	RMS	Media	Potencia Prom.	Desv. Est.
3	32	1.4142	0.6667	2.0000	1.2472
6	32	1.4142	0.6667	2.0000	1.2472
3	2	1.4142	0.6667	2.0000	1.2472
3	320	1.4142	0.6667	2.0000	1.2472
3	32	2.8285	1.3333	8.0003	2.4945

Ni la duplicación de muestras ni la variación en la frecuencia alteraron las métricas, confirmando que dependen únicamente del vector. Al escalar la entrada al doble (2,4,-2), la media y el RMS se duplicaron, mientras que la potencia se cuadruplicó (8,0), validando la respuesta del bloque.

III-D Aplicación propuesta: Detección de degradación en canal

El analizador estadístico permite monitorear en tiempo real la potencia promedio y el valor RMS de la señal recibida, funcionando como un indicador directo de la relación señal a ruido (SNR). Para evaluarlo, se sometió la señal a tres escenarios: canal limpio (ruido 0), moderado (0,1) y severo (0,5). Al aumentar el ruido,

el RMS y la potencia crecen de forma proporcional por superposición de energía. Así, definiendo un umbral sobre la señal limpia, el bloque detecta y alerta automáticamente sobre la degradación del canal.

IV CONCLUSIONES

- El uso de bloques embebidos en Python (*Embedded Python Blocks*) permite expandir las funciones de GNU Radio, facilitando el diseño de algoritmos DSP a la medida sin compilar módulos en C++.
- Se comprobó que el acumulador opera como un integrador recursivo en tiempo discreto, cuya amplitud de salida depende directamente de su memoria interna y de la persistencia de muestras con igual polaridad.
- Se demostró que el bloque diferenciador responde exclusivamente ante variaciones de amplitud, operando como un detector de flancos eficaz con salidas nulas ante entradas continuas y niveles fijos ante pendientes constantes.
- Se validó que los parámetros estadísticos son invariantes frente a cambios en la tasa de muestreo y dependen del vector de entrada, demostrando que el monitoreo conjunto de RMS y potencia promedio es un método confiable para detectar la degradación por ruido en un canal.

Referencias

- A. V. Oppenheim y R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.
- GNU Radio Project, "GNU Radio Documentation." [Online]. Available: <https://www.gnuradio.org>
- P. Z. Peebles, *Probability, Random Variables, and Random Signal Principles*, 4th ed. McGraw-Hill, 2001.